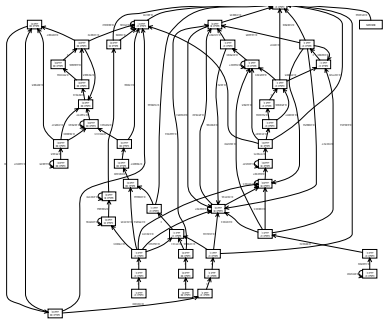
# Computational Biology

## Lecture 8: *Training HMMs*

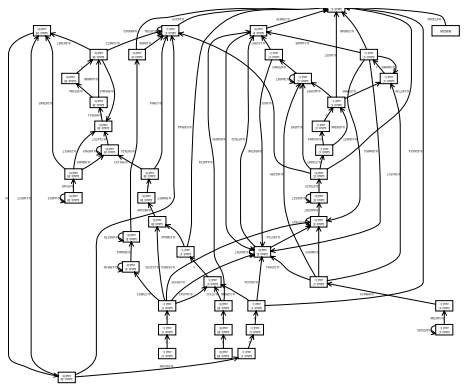


*Baum Welch, HMM Structures*



# Outline

1. **Recap**
2. Baum Welch
3. HMM Structure
4. Applications





# Recap

- HMMs model the generation of sequences
- They are probabilistic finite state automata describing the transition between state
- Each state emits a symbol with some probability
- We can compute the joint probability $\mathbb{P}(\boldsymbol{q}, \boldsymbol{\xi})$ of the states, $\boldsymbol{q}$, and sequence $\boldsymbol{\xi}$
- Using forward and backward variable we can sum over all possible paths through the states

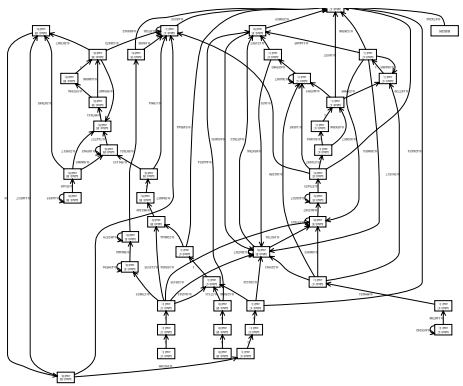


# Structure and Parameters

- The structure of the HMM is the set of states and the allowed transition between states
- Given a structure the parameters, $\boldsymbol{\theta}$, describes the emission probabilities, $e_i(\zeta)$ at each state and the transition probabilities, $a_{ij}$, between states
- When we understand the process generating the sequence very well then we can sometimes specify the structure and parameters *ab initio*
- Most of the time, we don't have a detailed understanding of the process and have to learn the parameters

# Outline

1. Recap
2. **Baum Welch**
3. HMM Structure
4. Applications





# Baum-Welch Training

- We can train the parameters of an HMM to maximise the likelihood of a set of training sequences
- This algorithm developed mainly by Leonard Baum is known as the Baum-Welch algorithm
- It is an iterative algorithm that finds a **local maximum** of the likelihood of a set of training sequences
- It is an example of an **expectation maximisation** (EM) strategy



# Iterative Improvement

- Given a set of sequence $\mathcal{D} = \{\boldsymbol{\xi}^\mu | \mu = 1, \ldots, P\}$, then if we knew the path through the set of states then we can count
  - $A_{ij}$ the number of times we transition from state $i$ to $j$
  - $E_i(\zeta)$ the number of times we emit symbol $\zeta$ when in state $i$
- To maximise the likelihood of the particular sequences we can set

$$e_i(\zeta) = \frac{E_i(\zeta)}{\sum\limits_{\xi \in \mathcal{A}} E_i(\xi)}, \qquad a_{ij} = \frac{A_{ij}}{\sum\limits_{k \in \mathcal{Q}} A_{ik}}$$



# Pseudo Counts

- A problem is that if we use the HMM on unseen data then there might be emissions or transitions that we've set to zero which we want to allow
- To solve that we can add **pseudo counts** giving some small probability of always emitting a symbol or making a transition

$$e_i(\zeta) = \frac{E_i(\zeta) + r_i(\zeta)}{\sum\limits_{\xi \in \mathcal{A}} (E_i(\xi) + r_i(\xi))}, \qquad a_{ij} = \frac{A_{ij} + r_{ij}}{\sum\limits_{k \in \mathcal{Q}} (A_{ik} + r_{ik})}$$

- Usually $r_i(\zeta)$ and $r_{ik}$ will be some small number

## Chicken and Egg

- The second problem we face though is we don't know the path through the states that a sequence should take
- That path will depend on the emission and transition probabilities (which is what we want to determine)
- We therefore use an iterative learning process where we initialise the transition and emission probabilities at random
- We then compute $E_i(\zeta)$ and $A_{ij}$ for those parameters
- We then update the parameters as described above and iterate, nudging the parameters towards a local optimum



## Computing the Emission

- For a given set of parameters $\boldsymbol{\theta}$ the probability of emitting a symbol $\zeta$ in state $i$ for the set of training sequences is

$$E_i(\zeta) = \sum_{\boldsymbol{\xi}^\mu \in \mathcal{D}} \sum_{t=1}^{T^\mu - 1} \mathbb{P}\left(q_t = i | \boldsymbol{\xi}^\mu, \boldsymbol{\theta}\right) \left[\!\left[ \xi_t^\mu = \zeta \right]\!\right]$$

- Where $[\![\text{predicate}]\!]$ equals 1 if predicate is true and 0 otherwise
- But from the backward and forward variables

$$E_i(\zeta) = \sum_{\boldsymbol{\xi}^\mu \in \mathcal{D}} \frac{1}{\mathbb{P}\left(\boldsymbol{\xi}^\mu | \boldsymbol{\theta}\right)} \sum_{t=1}^{T^\mu} \alpha_t^\mu(i)\, \beta_t^\mu(i) \left[\!\left[ \xi_t^\mu = \zeta \right]\!\right]$$



## Computing Transitions

- Similarly the expected number of transitions is given by

$$A_{ij} = \sum_{\boldsymbol{\xi}^\mu \in \mathcal{D}} \sum_{t=1}^{T^\mu} \mathbb{P}\left(q_t = i,\, q_{t+1} = j | \boldsymbol{\xi}^\mu, \boldsymbol{\theta}\right)$$

- After a bit of manipulations we find

$$A_{ij} = \sum_{\boldsymbol{\xi}^\mu \in \mathcal{D}} \frac{1}{\mathbb{P}\left(\boldsymbol{\xi}^\mu | \boldsymbol{\theta}\right)} \sum_{t=1}^{T^\mu - 1} \alpha_t^\mu(i)\, a_{ij}\, e_j(\xi_{t+1})\, \beta_{t+1}^\mu(j)$$

- We have already seen that we can compute $\mathbb{P}\left(\boldsymbol{\xi}^\mu | \boldsymbol{\theta}\right)$ from the forward variable

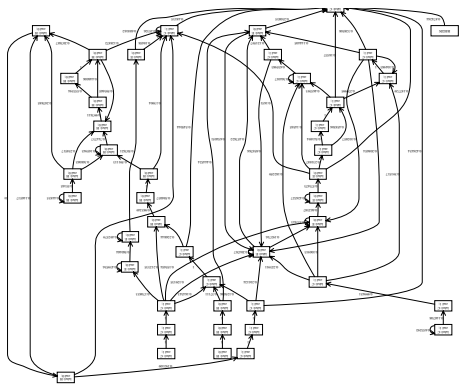


## Training in Practice

- Baum-Welch is widely used
- The implementation is slightly non-trivial (must prevent numbers underflowing), use sensible pseudo counts
- Most practitioners use packages (in python, R, C++, matlab)
- Also possible to use gradient based algorithms for parameter optimisation
- More complicated, but better convergence

# Outline

1. Recap
2. Baum Welch
3. **HMM Structure**
4. Applications





# HMM Structures

- Whereas training the parameters of an HMM is relatively straightforward designing the structure on an HMM is a black art
- We have to determine
  - the number of states
  - which transition we allow
- Sometimes we understand the problem enough to know what we are modelling, but this is rather unusual



# Number of States

- A state represents a part of sequence that has a different set of emission frequencies
- This should be a consequence of some physical differences
  - Coding area
  - Promoter region
  - Junk DNA
- If we have too few states we can fail to model significant differences
- If we have too many states then we learn chance fluctuations in emission frequencies giving poor generalisation



# Transitions

- The transitions should represent real physical structure
  - Promoter regions precede coding regions
- If we have a fully connected HMM we will get the best likelihood, but will tend to overfit
- Typically experts build quite structured network

# High-Order HMM

- The Markov property of HMMs often imposes too little short range structure
- We can generalise a Markov Chain to second order

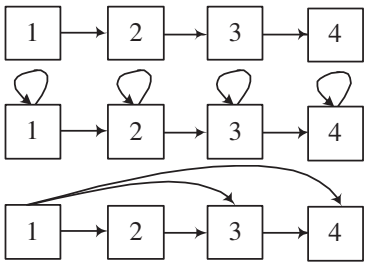
$$\mathbb{P}\left(x_i | x_{i-1},\, x_{i-2},\, \ldots, x_1\right) = \mathbb{P}\left(x_i | x_{i-1}, x_{i-2}\right)$$

- Can be further generalised to $n^{th}$-order
- In HMM's we often make the emission probabilities dependent on several previous elements in a sequence

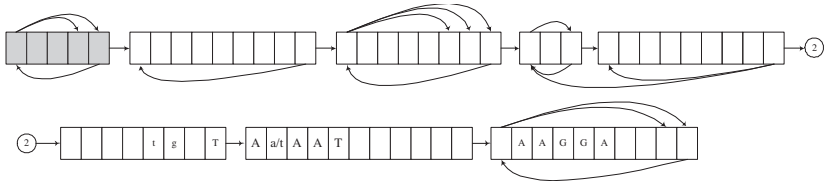


# Common Motifs

- In biology it is common to build an HMM from building blocks




- Use these to model promoter regions





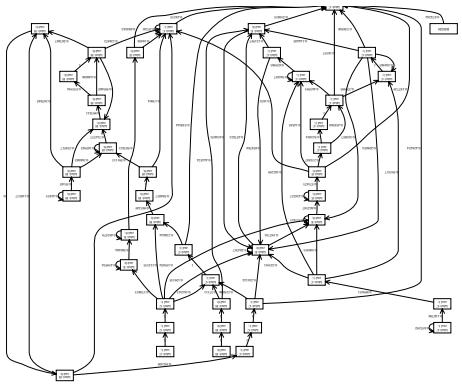

# Evolving HMMs

- Traditionally HMMs are designed by "experts"
- However by combining common motifs genetic algorithms can also be used
- For large problems the GA generated HMMs are actually significantly better than the "expert" HMMs
- It is probably rather hopeful to expect that a human can really design an HMM structure with tens of states



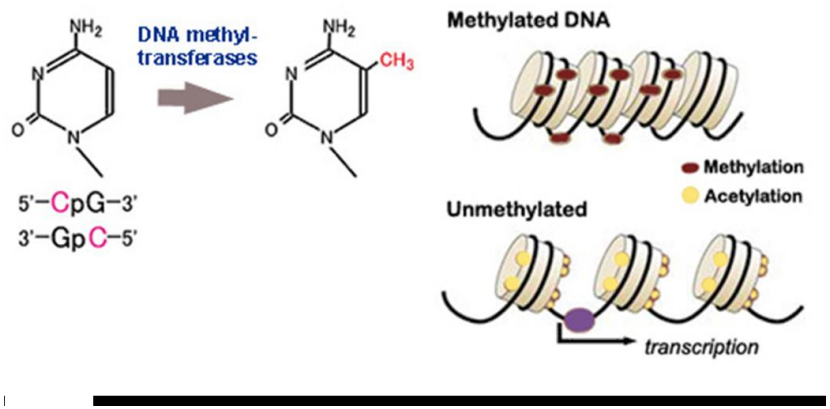
# Outline

1. Recap
2. Baum Welch
3. HMM Structure
4. **Applications**



# DNA Methylation

- Most biologists aren't that interested in dishonest casinos, but they are interested in promoter regions






# CpG Island

- In mammals non-methylated regions are associate with regions with an above average concentration of C followed by a G (known as CpG islands)
- These are usually under-represented because they are chemically modified by methylation (there is a relatively high chance of this methyl-C mutating to a T)
- However, this methylation is suppressed in some regions of the genome (e.g. in promoter regions)
- Therefore, finding CpG islands is of biological interest

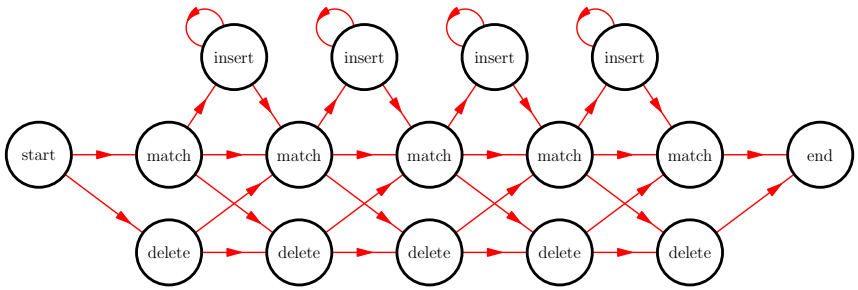


# Family Resemblance

- In the earlier lectures we said that one of the most important tasks was to identify similar proteins
- However, pairwise comparisons of proteins from distance relatives can give rather inconclusive results
- However, if we can compare against a whole family of proteins we often can spot similarities that are much less obvious
- A profile is an alignment of a whole family of proteins
- We can train a **profile HMM** to spot a distance relative

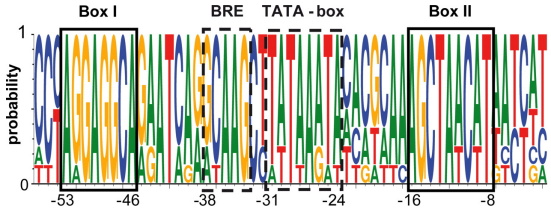


# Profile HMMs



# Gene Detection

- Gene detection is a significant challenge
- The first gene detectors (GENSCAN 1997) found only $\sim 10\%$ of genes in the human genome with huge number of false positive
- Over the years this has improved with new algorithms finding around 20 000 genes in the human genome
- Most algorithms use HMMs






# Summary

- To use HMMs in situations where we don't understand the generation process fully we train the parameters using Baum-Welch
- We still need to determine the architecture of HMMs
- Sometimes this is determined by the problem (e.g. profile HMM)
- Frequently we have to guess and experiment
- We can even use optimisation algorithms (although its not that easy)